



Trabajo Práctico N° 2
Validación y Verificación de Software
Unidad II — Testing Orientado a Objetos

FAI-3162
Cristopher Jason Ovaillos

Profesora: Rafaela Mazlou
Año: 2026

12 de mayo de 2026

Grupo	Grupo B
Dominio asignado	Sistema de Gestión de Flota Vehicular

Contenido

1. Introducción	5
2. Testing de Unidad OO — clase Auto	5
2.1. Código fuente provisto	5
2.2. Consigna 1: Invariante de clase	5
2.3. Consigna 2: Fixture pytest y justificación del scope	6
2.4. Consigna 3: Casos de prueba (patrón AAA)	6
2.5. Consigna 4: Verificación del invariante tras excepciones	8
2.6. Consigna 5: Medición de cobertura	8
3. FSM y Testing basado en Estado — Vehiculo de Flota	10
3.1. Consigna 6: Tabla de transición (Algoritmo 2.2)	10
3.2. Consigna 7: Diagrama de estados UML	11
3.3. Consigna 8: Casos de prueba C-Trans (Algoritmo 2.3) en JUnit 5	12
3.4. Consigna 9: Transiciones inválidas	14
3.5. Consigna 10: Cálculo de casos	17
4. Testing Polimórfico y LSP — jerarquía Vehiculo	18
4.1. Consigna 11: Implementá la Suite Heredada (Definición 2.11) como clase abstracta de test en Java. Debe contener un caso de prueba por cada combinación método × clase de equivalencia del contrato de la superclase.	18
4.2. Consigna 12: Aplicá el Algoritmo 2.4: ejecutá la Suite Heredada sobre cada subclase concreta (Auto, Moto, Camion) sustituyendo instancias.	19
4.3. Consigna 13: Analizá la subclase Moto y Camion: ¿violan el LSP o extienden el contrato de forma válida? Aplicá la Propiedad 2.3. Para Moto: el límite de 300 km en lugar de 500, ¿es <code>Pre_Moto</code> más fuerte o más débil que <code>Pre_Vehiculo</code> ? Para Camion: la precondition adicional de carga, ¿introduce una restricción que viola LSP?	20
4.4. Consigna 14: Para la subclase que extiende el contrato de forma válida: documentá formalmente la extensión (<code>Pre_D</code> , <code>Post_D</code> , <code>invariante_D</code>) y agregá los tests específicos de esa subclase que no están en la Suite Heredada.	20
4.5. Consigna 15: Aplicá la Propiedad 2.3: verificá para cada subclase que <code>Pre_D ⇒ Pre_C</code> y <code>Post_C ⇒ Post_D</code> . Si alguna subclase viola esta regla, indicá qué corrección de diseño aplicarías.	21
5. Partición por Categoría y Límites de Invariante — asignar()	21
5.1. Consigna 16: Aplica el Algoritmo 2.5 (Partición por Categoría) al método <code>asignar(legajo_conductor, km_estimados)</code> . Identifica todas las categorías relevantes (parámetros, estado del objeto, colaboradores si aplica) y sus particiones. Presenta el resultado en una tabla con columnas: Categoría Partición Tipo (válida/inválida) Representante.	21
5.2. Consigna 17. Identifica las restricciones entre particiones que reducen el catálogo completo. Calculá: (a) número de combinaciones del producto cartesiano sin restricciones; (b) número de casos resultantes tras aplicar las restricciones.	22
5.3. Consigna 18	23
5.4. Consigna 18(c): Estado del objeto y comportamiento esperado	24
5.5. Consigna 19: Codificá en pytest los casos de prueba derivados de los pasos 1 y 3. Organizalos en dos clases separadas: <code>TestParticionCategoria</code> y <code>TestLimitesInvariante</code> .	25
5.6. Consigna 20: Ejecutá la suite y presentá el reporte de cobertura. Identificá si alguna rama del método quedó sin cubrir y diseñá el caso adicional necesario.	27
6. Testing de Singleton — GestorFlota	29
6.1. Consigna 21: verificar el contrato de unicidad Algoritmo 2.8 paso 1: Dos llamadas a <code>GestorFlota.get_instancia()</code> deben retornar exactamente el mismo objeto. Usa <code>assert instancia 1 is instancia2</code> (identidad no igualdad)	29
6.2. Consigna 22. Implementá un fixture pytest con <code>scope='function'</code> que invoque <code>_reset_para_tests()</code> en el teardown. Explicá por qué es necesario este reset y qué problema de contaminación de estado se evita (Algoritmo 2.8, Paso 3).	29

6.3.	Demostrá el problema de contaminación sin el reset: escribí dos tests donde el segundo falla porque el primero dejó estado acumulado en el singleton. Luego corregí usando el fixture del punto anterior.	30
6.4.	Verificá thread-safety del Singleton: usá el módulo threading de Python para lanzar 50 hilos que llamen <code>get_instancia()</code> concurrentemente. Verificá que todas las instancias retornadas son idénticas (Algoritmo 2.8, Paso 4).	31
6.5.	Consigna 25: Analizá el método <code>asignar()</code> del <code>GestorFlota</code> desde la perspectiva del testing: ¿es un buen diseño que el Singleton delegue en los objetos <code>Vehiculo</code> ? ¿Cómo simplifica o complica el testing la existencia del Singleton como intermediario? . . .	32

Lista de figuras

1.	Resultado del test 3.c en pytest. Se observa que se lanza la excepción esperada y el estado del vehículo no cambia.	8
2.	Resultado de aplicar <code>-v -cov=flota -cov-branch</code> a los tests de <code>auto.py</code> . Se observa que se alcanzó un C0 del 87,23 % y un C1 del 63,64 %.	9
3.	Tabla de transición de la FSM para Vehículo (Algoritmo 2.2). Cada celda indica el estado destino y efecto si la transición es válida, o la excepción si no lo es.	10
4.	Diagrama de estados para Vehículo. Flechas sólidas: transiciones válidas. Flechas punteadas rojas: transiciones inválidas (excepción).	11
5.	Contrato de cada metodo de Vehículo.	18
6.	Ejemplo de ejecución de la suite heredada sobre las subclases (solo errores).	20
7.	Resultado del test de unicidad de instancia.	29
8.	Resultado del test de contaminación de estado.	30
9.	Resultado del test de contaminación de estado corregido.	31
10.	Resultado del test de thread-safety del singleton.	31

Lista de tablas

1.	Invariantes de clase para <code>Auto</code>	6
2.	Resultados de cobertura — <code>auto.py</code>	9
3.	Sentencias no cubiertas en <code>auto.py</code>	10
4.	clases de equivalencia - <code>Vehiculo.asignar(legajo, km)</code>	19
5.	Casos de prueba - <code>Vehiculo.asignar()</code> y <code>Vehiculo.liberar()</code>	19
6.	Tabla de categorías y particiones para el método <code>asignar()</code>	22
7.	Límites de entrada para <code>asignar()</code> y comportamiento esperado	25

1. Introducción

El presente trabajo práctico tiene como objetivo aplicar las técnicas de testing orientado a objetos sobre un sistema de gestión de flota vehicular. Este dominio administra vehículos de distintos tipos (automóviles, motos, camiones) con un ciclo de vida controlado por una máquina de estados finita (algoritmo 2.6).

2. Testing de Unidad OO — clase Auto

2.1. Código fuente provisto

El sistema de gestión de flota vehicular administra vehículos de distintos tipos. La clase `Auto` hereda de `Vehiculo` (abstracta) y representa automóviles de la flota. El código provisto contiene defectos intencionales que los tests deben detectar.

```
# flota/auto.py
from enum import Enum

class EstadoVehiculo(Enum):
    DISPONIBLE = 'disponible'
    EN_USO = 'en_uso'
    MANTENIMIENTO = 'mantenimiento'
    BAJA = 'baja'

class Auto:
    def __init__(self, patente: str, marca: str, anio: int,
                  km_actuales: float, num_puertas: int):
        if not patente or len(patente) < 6:
            raise ValueError('Patente inválida')
        if km_actuales < 0:
            raise ValueError('km_actuales no puede ser negativo')
        if num_puertas not in (2, 4, 5):
            raise ValueError('num_puertas debe ser 2, 4 o 5')
        self._patente = patente
        self._marca = marca
        self._anio = anio
        self._km = km_actuales
        self._puertas = num_puertas
        self._estado = EstadoVehiculo.DISPONIBLE
        self._conductor = None
```

2.2. Consigna 1: Invariante de clase

La invariante de clase es un predicado lógico sobre los atributos de una clase que debe ser verdadera al finalizar todo constructor y al finalizar todo método público, siempre que también fuera verdadero al inicio de dicho método. La invariante no necesita cumplirse durante la ejecución interna de un método, sino en sus puntos de entrada y salida observables.

Segun esto, podemos identificar a las invariantes de clase por que son aquellas a las cuales debemos capturar todas las condiciones (es decir, variables) cuanto esta es verdadera para cualquier objeto valido, independiente del metodo que se ejecute.

Para la clase `Auto`, identificamos las siguientes invariantes. Como se observa en la tabla 1: donde $S = \{\text{DISPONIBLE}, \text{EN_USO}, \text{MANTENIMIENTO}, \text{BAJA}\}$.

El invariante completo funciona como una condicion global del objeto `Auto`, es decir, debe cumplirse para cualquier instancia válida de la clase. Por lo tanto, el invariante de clase para `Auto` se expresa como:

$$I(\text{Auto}) = \text{patente} \neq '' \wedge \text{len}(\text{patente}) \geq 6 \wedge \text{km} \geq 0 \wedge \text{puertas} \in \{2, 4, 5\} \wedge \text{estado} \in S \wedge (\text{estado} = \text{EN_USO} \Rightarrow \text{conductor} \neq \text{None})$$

El algoritmo AAA es adaptable en un contexto OO. Donde la fase **Arrange** incluye la construcción del objeto bajo prueba y la configuración de su estado inicial. Este ejercicio se empleara

Tabla 1: Invariantes de clase para **Auto**

Atributo	Invariante	Predicado lógico
<code>_patente</code>	No vacío, longitud ≥ 6	$\text{patente} \neq "" \wedge \text{len}(\text{patente}) \geq 6$
<code>_km</code>	No negativo	$\text{km_actuales} \geq 0$
<code>_puertas</code>	Valores permitidos: 2, 4, 5	$\text{num_puertas} \in \{2, 4, 5\}$
<code>_estado</code>	Estado válido del enum	$\text{estado} \in S$
<code>_conductor</code>	Consistencia con estado	$(\text{estado} = \text{EN_USO} \Rightarrow \text{conductor} \neq \text{None}) \wedge (\text{estado} \neq \text{EN_USO} \Rightarrow \text{conductor} = \text{None})$

el scope de **function** para instanciar un nuevo objeto para cada test, garantizando que cada caso parte de un estado conocido y reproducible, tal como establece el Algoritmo 2.1 y la Definición 2.5 sobre fixtures de prueba.

fuelle: [pytest fixtures](#)

fuelle: [pytest](#)

“**function**: the default scope, the fixture is destroyed at the end of the test.”

2.3. Consigna 2: Fixture **pytest** y justificación del scope

Algoritmo AAA es adaptable a un contexto orientado a objetos, donde la fase ARRANGE ahora incluye la construcción del objeto bajo prueba y la configuración de su estado inicial (invariante obtenida anteriormente). Con ayuda de fixture de **pytest** para plantear un objeto `auto_valido` con atributos que cumplen el invariante de clase, se garantiza que cada test parte de un estado conocido y reproducible, tal como establece el Algoritmo 2.1 y la Definición 2.5 sobre fixtures de prueba.

Con ayuda de <https://docs.pytest.org/en/6.2.x/fixture.html> documentación de **pytest** (fixture) que dice: “function: the default scope, the fixture is destroyed at the end of the test”, se justifica el uso del scope **function** para la fixture `auto_valido`, ya que cada test debe trabajar con una instancia fresca del objeto para evitar efectos colaterales entre tests y garantizar la independencia de cada caso de prueba.

Para agregar, tenemos la siguiente información:

- **cada caso parte de un estado conocido y reproducible**
- Esto se conecta con la Definición 2.5 de fixtures de prueba, que dice lo siguiente:

“garantiza que cada caso parte de un estado conocido y reproducible independientemente del orden”

2.4. Consigna 3: Casos de prueba (patrón AAA)

A partir del ejercicio 3.c se uso <https://docs.pytest.org/en/stable/how-to/assert.html> para escribir y reportar `assert` con ayuda de **pytest**.

3.a) asignar con `km_estimados=1.0` (límite inferior, válido)

```
def test_ejercicio_3a(auto_valido):
    # Arrange
    legajo = "charmander"
    km = 1.0 # pre: (0,500] válido
    # Act
    auto_valido.asignar(legajo, km)
    # Assert
    assert auto_valido.get_estado() == EstadoVehiculo.EN_USO
    assert auto_valido._conductor == legajo
    assert auto_valido.get_km() >= 0 # invariante
```

Resultado: Test pasado. El vehículo cambia a `EN_USO` y el conductor queda registrado.

3.b) asignar con km_estimados=500.0 (límite superior) — BUG detectado

La precondition indica que el rango válido es (0,500], por lo que 500.0 debería ser aceptado. Sin embargo, el código contiene un bug en la línea 30:

```
if km_estimados <= 0 or km_estimados >= 500: # BUG
```

La condición `>= 500` rechaza el valor 500.0, que según la precondition debería ser válido.

- **Resultado esperado:** Vehículo pasa a estado EN_USO con conductor registrado.
- **Resultado real:** `ValueError('km_estimados fuera de rango')`
- **Corrección propuesta:** Cambiar `>= 500` por `> 500`.

3.c) asignar con km_estimados=0.0 (borde inválido)

```
def test_ejercicio3c(auto_valido):  
    estado_inicial = auto_valido.get_estado()  
    km_inicial = auto_valido.get_km()  
    with pytest.raises(ValueError):  
        auto_valido.asignar("caterpie", 0.0)  
    assert auto_valido.get_estado() == estado_inicial  
    assert auto_valido.get_km() >= km_inicial
```

Resultado: Test pasado. Se lanza `ValueError` y el estado permanece DISPONIBLE.

3.d) asignar cuando estado=EN_USO

```
def test_ejercicio3d(auto_valido):  
    auto_valido.asignar("metapod", 100)  
    estado_inicial = auto_valido.get_estado()  
    km_inicial = auto_valido.get_km()  
    with pytest.raises(PermissionError):  
        auto_valido.asignar("metapod", 50)  
    assert auto_valido.get_estado() == estado_inicial  
    assert auto_valido.get_km() == km_inicial
```

Resultado: Test pasado. Se lanza `PermissionError` y el estado se mantiene en EN_USO.

3.e) liberar con km_recorridos=150.0

```
def test_ejercicio3e(auto_valido):  
    auto_valido.asignar("patamon", 200)  
    km_inicial = auto_valido.get_km()  
    auto_valido.liberar(150.0)  
    assert auto_valido.get_estado() == EstadoVehiculo.DISPONIBLE  
    assert auto_valido._conductor is None  
    assert auto_valido.get_km() == km_inicial + 150.0  
    assert auto_valido.get_km() >= 0 # invariante
```

Resultado: Test pasado. El vehículo vuelve a DISPONIBLE, conductor `None`, y los km se acumulan correctamente.

3.f) liberar cuando estado=DISPONIBLE

```
def test_ejercicio3f(auto_valido):  
    estado_inicial = auto_valido.get_estado()  
    km_inicial = auto_valido.get_km()  
    with pytest.raises(PermissionError):  
        auto_valido.liberar(100)  
    assert auto_valido.get_estado() == estado_inicial  
    assert auto_valido.get_km() == km_inicial
```

Resultado: Test pasado. Se lanza `PermissionError` y el estado permanece DISPONIBLE.

3.g) enviar_mantenimiento desde DISPONIBLE

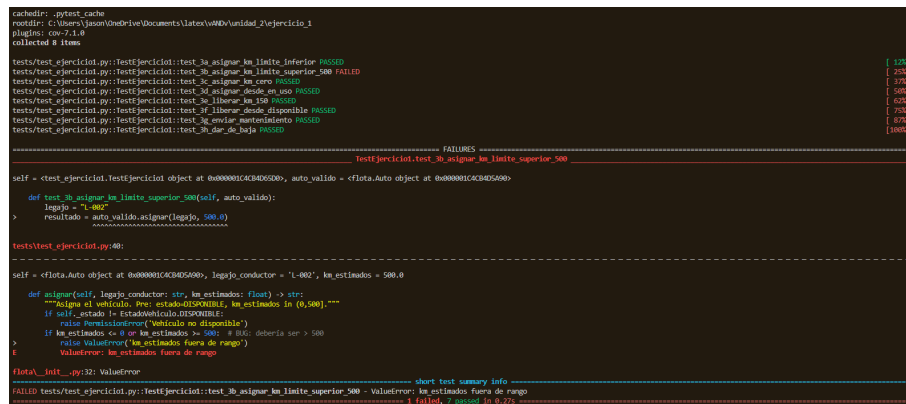
```
def test_ejercicio_3g(auto_valido):
    auto_valido.enviar_mantenimiento()
    assert auto_valido.get_estado() == EstadoVehiculo.MANTENIMIENTO
```

Resultado: Test pasado.

3.h) dar_de_baja desde MANTENIMIENTO

```
def test_ejercicio3h(auto_valido):
    auto_valido.enviar_mantenimiento()
    auto_valido.dar_de_baja()
    assert auto_valido.get_estado() == EstadoVehiculo.BAJA
```

Resultado: Test pasado.



```
cachedir: .pytest_cache
rootdir: C:\Users\jason\OneDrive\Documents\lato\veh\Unidad_2\ejercicio_1
plugins: cov-2.1.0
collected 8 items

tests/test_ejercicio.py::TestEjercicio::test_3a_asignar_km_limite_inferior PASSED [ 12%]
tests/test_ejercicio.py::TestEjercicio::test_3b_asignar_km_limite_superior_500 FAILED [ 25%]
tests/test_ejercicio.py::TestEjercicio::test_3c_asignar_km_cero PASSED [ 37%]
tests/test_ejercicio.py::TestEjercicio::test_3d_asignar_estado_en_uso PASSED [ 50%]
tests/test_ejercicio.py::TestEjercicio::test_3e_liberar_km_150 PASSED [ 62%]
tests/test_ejercicio.py::TestEjercicio::test_3f_liberar_estado_disponible PASSED [ 75%]
tests/test_ejercicio.py::TestEjercicio::test_3g_enviar_mantenimiento PASSED [ 87%]
tests/test_ejercicio.py::TestEjercicio::test_3h_dar_de_baja PASSED [100%]

===== FAILURES =====
test_ejercicio.py::TestEjercicio::test_3b_asignar_km_limite_superior_500
self = <test_ejercicio.py::TestEjercicio object at 0x000001C6B4D5000>, auto_valido = <flota.Auto object at 0x000001C6B4D5000>

    def test_3b_asignar_km_limite_superior_500(self, auto_valido):
        legajo = "L-002"
        resultado = auto_valido.asignar(legajo, 500.0)
        # resultado = auto_valido.asignar(legajo, 500.0)

tests/test_ejercicio.py:40:
-----
self = <flota.Auto object at 0x000001C6B4D5000>, legajo_conductor = "L-002", km_estimados = 500.0

    def asignar(self, legajo_conductor: str, km_estimados: float) -> str:
        """Registra el vehículo. Pre: estado=DISPONIBLE, km_estimados <= (0,500)."""
        if self.estado != EstadoVehiculo.DISPONIBLE:
            raise PermissionError("Vehículo no disponible")
        if km_estimados <= 0 or km_estimados >= 500: # dato: debería ser > 500
            raise ValueError("km_estimados fuera de rango")
        # ValueError: km_estimados fuera de rango

flota/_init_.py:32: ValueError
===== short test summary info =====
FAILED tests/test_ejercicio.py::TestEjercicio::test_3b_asignar_km_limite_superior_500 - ValueError: km_estimados fuera de rango
1 failed, 7 passed in 0.27s
```

Figura 1: Resultado del test 3.c en pytest. Se observa que se lanza la excepción esperada y el estado del vehículo no cambia.

2.5. Consigna 4: Verificación del invariante tras excepciones

Para cada caso que espera una excepción (3.c, 3.d, 3.f), se incluyó un assert adicional que verifica que el estado del objeto no cambió tras la excepción:

- **3.c:** Tras `ValueError` por `km=0.0`, el estado permanece `DISPONIBLE` y los `km` no varían.
- **3.d:** Tras `PermissionError` por estado `EN_USO`, el estado se mantiene en `EN_USO` y los `km` no cambian.
- **3.f:** Tras `PermissionError` por estado `DISPONIBLE`, el estado permanece `DISPONIBLE` y los `km` no cambian.

2.6. Consigna 5: Medición de cobertura

Se ejecutó `pytest -cov -branch` para medir la cobertura. Los resultados obtenidos son:

Con ayuda de la Unidad N° 1 (definición 4.2 y 4.3) sabemos que C0 toda sentencia del programa es ejecutada por al menos un caso de prueba (sentencias ejecutadas/total sentencia)*100

Ramas no cubiertas

Las ramas parciales (líneas 11, 13, 15, 41, 50, 56, 61) corresponden a condiciones donde solo se ejecutó la rama `False` (la condición no se cumplió), pero nunca la rama `True`. Por ejemplo, en la línea 11 `if not patente or len(patente) < 6`, siempre se pasó una patente válida, por lo que la rama que lanza la excepción nunca se tomó.

Los tests implementados cubren los caminos principales de cada método y los casos de error especificados en las consignas, logrando un C0 de 87,23 %. El C1 de 63,64 % indica que hay ramas

```

===== FAILURES =====
TestEjercicio1.test_3b_asignar_km_limite_superior_500

self = <test_ejercicio1.TestEjercicio1 object at 0x000002B507B54910>, auto_valido = <flota.Auto object at 0x000002B507B551D0>

    def test_3b_asignar_km_limite_superior_500(self, auto_valido):
        legajo = "L-002"
        resultado = auto_valido.asignar(legajo, 500.0)
        ~~~~~
tests/test_ejercicio1.py:48:

self = <flota.Auto object at 0x000002B507B551D0>, legajo_conductor = 'L-002', km_estimados = 500.0

    def asignar(self, legajo conductor: str, km_estimados: float) -> str:
        """Asigna el vehiculo. Pre: estado=DISPONIBLE, km_estimados in (0,500]."""
        if self._estado != EstadoVehiculo.DISPONIBLE:
            raise PermissionError('Vehiculo no disponible')
        if km_estimados <= 0 or km_estimados >= 500: # BUG: debería ser > 500
            raise ValueError('km_estimados fuera de rango')
        ~~~~~
Flota/_init_.py:32: ValueError

===== tests coverage =====
coverage: platform win32, python 3.14.2-final-0

Name                               Stmts   Miss Branch BrPart  Cover   Missing
-----
Flota/_init_.py                     47      6     22      7    81%   14, 16, 18, 42, 50, 55, 58->59
TOTAL                               47      6     22      7    81%

===== short test summary info =====
FAILED tests/test_ejercicio1.py::TestEjercicio1::test_3b_asignar_km_limite_superior_500 - ValueError: km_estimados fuera de rango
===== 1 failed, 7 passed in 0.18s =====
(.venv) PS C:\Users\jason\OneDrive\Documents\latex\WAND\unidad_2\ejercicio_1>

```

Figura 2: Resultado de aplicar `-v -cov=flota -cov-branch` a los tests de `auto.py`. Se observa que se alcanzó un C0 del 87,23 % y un C1 del 63,64 %.

Tabla 2: Resultados de cobertura — `auto.py`

Métrica	Valor	
Sentencias totales (Stmts)	47	
Sentencias ejecutadas	41	
Sentencias no ejecutadas (Miss)	6	
C0 (cobertura de sentencias)	87,23 %	$(41/47 \times 100)$
C1 (cobertura de ramas)	63,64 %	$(8/22 \times 100)$
Ramas totales	22	
Ramas no cubiertas	8	
Ramas parciales	7	

de validación de entrada en el constructor y de métodos que no fueron ejercitadas porque los casos de prueba se centraron en los escenarios de asignación/liberación y transiciones de estado definidos en la consigna.

Tabla 3: Sentencias no cubiertas en `auto.py`

Línea	Sentencia	Razón
12	<code>raise ValueError('Patente inválida')</code>	No se testeó con patente vacía o <code>len < 6</code>
14	<code>raise ValueError('km_actuales < 0')</code>	No se testeó con <code>km_actuales < 0</code>
16	<code>raise ValueError('num_puertas debe ser 2, 4 o 5')</code>	No se testeó con <code>num_puertas ∉ {2, 4, 5}</code>
42	<code>raise ValueError('km_recorridos < 0')</code>	No se testeó <code>liberar()</code> con <code>km_recorridos < 0</code>
51	<code>raise PermissionError('...mantenimiento en uso')</code>	No se testeó <code>enviar_mantenimiento()</code> desde <code>EN_USO</code>
57	<code>raise PermissionError('...baja...en uso')</code>	No se testeó <code>dar_de_baja()</code> desde <code>EN_USO</code>

3. FSM y Testing basado en Estado — Vehículo de Flota

3.1. Consigna 6: Tabla de transición (Algoritmo 2.2)

Aplicamos el **Algoritmo 2.2** paso a paso:

Paso 1. Identificar los estados. Según la especificación verbal y la Definición 2.6, los estados son los grupos de valores de atributos que producen comportamientos cualitativamente distintos:

$$S = \{\text{DISPONIBLE}, \text{EN_USO}, \text{MANTENIMIENTO}, \text{BAJA}\}$$

Paso 2. Identificar el estado inicial. El objeto queda en `DISPONIBLE` inmediatamente después del constructor:

$$s_0 = \text{DISPONIBLE}$$

Paso 3. Para cada método público y cada estado, determinar si es aplicable. Los eventos (métodos que modifican el estado) son:

$$\Sigma = \{\text{asignar}(), \text{liberar}(), \text{enviarMantenimiento}(), \text{darDeBaja}()\}$$

estado / evento	asignar	liberar	enviarMantenimiento	darDeBaja
DISPONIBLE	EN_USO	pre=en_uso, no se cumple	<u>EN_MANTENIMIENTO</u>	BAJA
EN_USO	ILEGAL	<u>DISPONIBLE</u>	<u>EN_MANTENIMIENTO</u>	ILEGAL
MANTENIMIENTO	ILEGAL	DISPONIBLE	ILEGAL	BAJA
BAJA	ILEGAL	ILEGAL	ILEGAL	ILEGAL

Figura 3: Tabla de transición de la FSM para Vehículo (Algoritmo 2.2). Cada celda indica el estado destino y efecto si la transición es válida, o la excepción si no lo es.

Justificación de cada celda según la especificación:

- **DISPONIBLE + asignar():** Válida. El vehículo puede ser asignado a un conductor. Transición `DISPONIBLE → EN_USO`.
- **DISPONIBLE + liberar():** Inválida. No se puede liberar un vehículo que no está en uso. Lanza `PermissionError`.
- **DISPONIBLE + enviarMantenimiento():** Válida. Se puede enviar a mantenimiento preventivo. Transición `DISPONIBLE → MANTENIMIENTO`.

- **DISPONIBLE + darDeBaja():** Válida. Se puede dar de baja desde cualquier estado excepto EN_USO. Transición `DISPONIBLE → BAJA`.
- **EN_USO + asignar():** Inválida. Ya está asignado. Lanza `PermissionError`.
- **EN_USO + liberar():** Válida. Finaliza el viaje. Transición `EN_USO → DISPONIBLE`.
- **EN_USO + enviarMantenimiento():** Válida. Mantenimiento correctivo por avería. Transición `EN_USO → MANTENIMIENTO`.
- **EN_USO + darDeBaja():** Inválida. No se puede dar de baja un vehículo en uso. Lanza `PermissionError`.
- **MANTENIMIENTO + asignar():** Inválida. Está en el taller. Lanza `PermissionError`.
- **MANTENIMIENTO + liberar():** Válida. Finaliza la revisión y vuelve a disponible. Transición `MANTENIMIENTO → DISPONIBLE`.
- **MANTENIMIENTO + enviarMantenimiento():** Inválida. Ya está en mantenimiento. Lanza `PermissionError`.
- **MANTENIMIENTO + darDeBaja():** Válida. Se puede dar de baja desde mantenimiento. Transición `MANTENIMIENTO → BAJA`.
- **BAJA + cualquier operación:** Todas inválidas. El vehículo fue retirado definitivamente. Lanza `IllegalStateException`.

3.2. Consigna 7: Diagrama de estados UML

Se construye a partir de la tabla de transición.

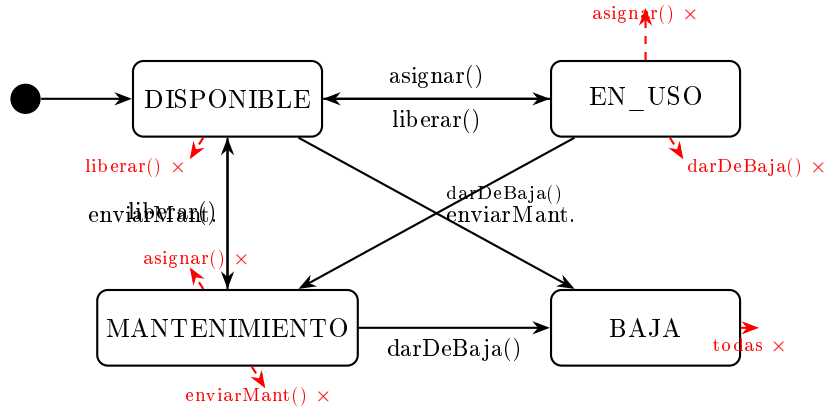


Figura 4: Diagrama de estados para Vehículo. Flechas sólidas: transiciones válidas. Flechas punteadas rojas: transiciones inválidas (excepción).

Elementos del diagrama:

- **Nodo inicial:** Círculo negro apuntando a `DISPONIBLE`, indicando que es el estado tras la construcción del objeto.
- **Estados:** Rectángulos para cada estado de la FSM.
- **Transiciones válidas:** Flechas etiquetadas con el evento (método) que las dispara.
- **Transiciones inválidas:** Flechas punteadas indicando que la operación lanza excepción y el estado no cambia.
- **BAJA** es un estado que no tiene transiciones salientes válidas.

3.3. Consigna 8: Casos de prueba C-Trans (Algoritmo 2.3) en JUnit 5

Aplicamos el **Algoritmo 2.3** paso a paso:

Paso 1. La FSM y su tabla de transición ya fueron construidas (Consigna 6).

Paso 2. Listar todas las transiciones válidas y marcarlas como NO CUBIERTAS:

- **T1:** DISPONIBLE → EN_USO asignar()
- **T2:** DISPONIBLE → MANTENIMIENTO enviarMantenimiento()
- **T3:** DISPONIBLE → BAJA darDeBaja()
- **T4:** EN_USO → DISPONIBLE liberar()
- **T5:** EN_USO → MANTENIMIENTO enviarMantenimiento()
- **T6:** MANTENIMIENTO → DISPONIBLE liberar()
- **T7:** MANTENIMIENTO → BAJA darDeBaja()

Son **7 transiciones válidas** en total.

Paso 3. Para cada transición, diseñar la secuencia que lleve al objeto desde s_0 hasta el estado origen:

- **T1, T2, T3:** El estado origen es DISPONIBLE, que es el estado inicial s_0 . No se necesita secuencia previa; basta con `new Vehiculo()`.
- **T4, T5:** El estado origen es EN_USO. Secuencia: `new Vehiculo() → asignar()`.
- **T6, T7:** El estado origen es MANTENIMIENTO. Secuencia: `new Vehiculo() → enviarMantenimiento()` (desde DISPONIBLE).

Paso 4-6. Implementación completa en JUnit 5:

Listing 1: TestVehiculoTransiciones.java — Cobertura C-Trans completa

```
1 import static org.junit.jupiter.api.Assertions.*;
2 import org.junit.jupiter.api.BeforeEach;
3 import org.junit.jupiter.api.Test;
4
5 class TestVehiculoTransiciones {
6
7     private Vehiculo vehiculo;
8
9     @BeforeEach
10    void setUp() {
11        //cada test comienza con un vehiculo nuevo en DISPONIBLE
12        vehiculo = new Vehiculo("ABC123", "Toyota", 2020);
13        assertEquals(EstadoVehiculo.DISPONIBLE, vehiculo.getEstado());
14    }
15
16    // trans. validas
17
18    //T1
19    @Test
20    void asignar_desde_disponible_cambia_a_en_uso() {
21        //DISPONIBLE- a EN_USO via asignar(legajo, km)
22        //ARRANGE: vehiculo ya esta en DISPONIBLE
23        String legajo = "L-001";
24        double kmEstimados = 100.0;
25        //ACT
26        String resultado = vehiculo.asignar(legajo, kmEstimados);
27        //ASSERT: estado destino + efecto + invariante
28        assertEquals(EstadoVehiculo.EN_USO, vehiculo.getEstado());
29        assertNotNull(resultado);
30        assertTrue(resultado.contains(legajo));
31    }
```

```

31         assertTrue(vehiculo.getKm() >= 0); // invariante
32     }
33
34     //T2
35     @Test
36     void enviarMantenimiento_desde_disponible_cambia_a_mantenimiento() {
37         //DISPONIBLE -> MANTENIMIENTO enviarMantenimiento()
38         //ARRANGE: vehiculo ya esta en DISPONIBLE (setUp)
39         // ACT
40         vehiculo.enviarMantenimiento();
41         // ASSERT
42         assertEquals(EstadoVehiculo.MANTENIMIENTO, vehiculo.getEstado());
43         assertTrue(vehiculo.getKm() >= 0); // invariante
44     }
45
46     //T3
47     @Test
48     void darDeBaja_desde_disponible_cambia_a_baja() {
49         //DISPONIBLE -> BAJA darDeBaja()
50         //ARRANGE: vehiculo ya esta en DISPONIBLE
51         //ACT
52         vehiculo.darDeBaja();
53         //ASSERT
54         assertEquals(EstadoVehiculo.BAJA, vehiculo.getEstado());
55         assertTrue(vehiculo.getKm() >= 0); // invariante
56     }
57
58     //T4
59     @Test
60     void liberar_desde_en_uso_cambia_a_disponible() {
61         //EN_USO -> DISPONIBLE via liberar(kmRec)
62         //ARRANGE: llevar a EN_USO
63         vehiculo.asignar("L-002", 200.0);
64         assertEquals(EstadoVehiculo.EN_USO, vehiculo.getEstado());
65         double kmAntes = vehiculo.getKm();
66
67         // ACT
68         vehiculo.liberar(150.0);
69
70         // ASSERT: estado destino + efecto + invariante
71         assertEquals(EstadoVehiculo.DISPONIBLE, vehiculo.getEstado());
72         assertEquals(kmAntes + 150.0, vehiculo.getKm(), 0.001);
73         assertTrue(vehiculo.getKm() >= 0); // invariante
74     }
75
76     //T5
77     @Test
78     void enviarMantenimiento_desde_en_uso_cambia_a_mantenimiento() {
79         //EN_USO -> MANTENIMIENTO via enviarMantenimiento()
80         //ARRANGE: llevar a EN_USO
81         vehiculo.asignar("L-003", 50.0);
82         assertEquals(EstadoVehiculo.EN_USO, vehiculo.getEstado());
83
84         // ACT
85         vehiculo.enviarMantenimiento();
86
87         // ASSERT
88         assertEquals(EstadoVehiculo.MANTENIMIENTO, vehiculo.getEstado());
89         assertTrue(vehiculo.getKm() >= 0); // invariante
90     }
91
92     //T6
93     @Test

```

```

94 void liberar_desde_mantenimiento_cambia_a_disponible() {
95     //MANTENIMIENTO -> DISPONIBLE liberar(kmRec)
96     //ARRANGE: llevar a MANTENIMIENTO
97     vehiculo.enviarMantenimiento();
98     assertEquals(EstadoVehiculo.MANTENIMIENTO, vehiculo.getEstado());
99     double kmAntes = vehiculo.getKm();
100
101     // ACT
102     vehiculo.liberar(0.0);
103
104     // ASSERT
105     assertEquals(EstadoVehiculo.DISPONIBLE, vehiculo.getEstado());
106     assertEquals(kmAntes, vehiculo.getKm(), 0.001); // no se suman km
107     assertTrue(vehiculo.getKm() >= 0); // invariante
108 }
109
110 //T7
111 @Test
112 void darDeBaja_desde_mantenimiento_cambia_a_baja() {
113     //MANTENIMIENTO -> BAJA via darDeBaja()
114     //ARRANGE: llevar a MANTENIMIENTO
115     vehiculo.enviarMantenimiento();
116     assertEquals(EstadoVehiculo.MANTENIMIENTO, vehiculo.getEstado());
117
118     // ACT
119     vehiculo.darDeBaja();
120
121     // ASSERT
122     assertEquals(EstadoVehiculo.BAJA, vehiculo.getEstado());
123     assertTrue(vehiculo.getKm() >= 0); // invariante
124 }
125 }

```

3.4. Consigna 9: Transiciones inválidas

Para todas las transiciones inválidas de la tabla. Según el Algoritmo 2.3, Paso 5: para cada celda con excepción, verificar que se lanza la excepción correcta y que el estado del objeto no cambia. Las transiciones inválidas son:

- INV1: DISPONIBLE + liberar() → PermissionError
- INV2: EN_USO + asignar() → PermissionError
- INV3: EN_USO + darDeBaja() → PermissionError
- INV4: MANTENIMIENTO + asignar() → PermissionError
- INV5: MANTENIMIENTO + enviarMantenimiento() → PermissionError
- INV6: BAJA + asignar() → IllegalStateException
- INV7: BAJA + liberar() → IllegalStateException
- INV8: BAJA + enviarMantenimiento() → IllegalStateException
- INV9: BAJA + darDeBaja() → IllegalStateException

Son 9 transiciones inválidas en total.

Listing 2: TestVehiculoTransicionesInvalidas.java

```

1 class TestVehiculoTransicionesInvalidas {
2
3     private Vehiculo vehiculo;
4

```

```

5     @BeforeEach
6     void setUp() {
7         vehiculo = new Vehiculo("ABC123", "Toyota", 2020);
8     }
9
10    //INV1
11    @Test
12    void liberar_desde_disponible_lanza_permission_error() {
13        //DISPONIBLE
14        EstadoVehiculo estadoInicial = vehiculo.getEstado();
15        double kmInicial = vehiculo.getKm();
16        assertEquals(EstadoVehiculo.DISPONIBLE, estadoInicial);
17
18        assertThrows(PermissionError.class,
19            () -> vehiculo.liberar(100.0));
20
21        // Verificar que el estado no cambio
22        assertEquals(estadoInicial, vehiculo.getEstado());
23        assertEquals(kmInicial, vehiculo.getKm(), 0.001);
24        assertTrue(vehiculo.getKm() >= 0); // invariante
25    }
26
27    //INV2
28    @Test
29    void asignar_desde_en_uso_lanza_permission_error() {
30        vehiculo.asignar("L-010", 100.0);
31        EstadoVehiculo estadoInicial = vehiculo.getEstado();
32        double kmInicial = vehiculo.getKm();
33        assertEquals(EstadoVehiculo.EN_USO, estadoInicial);
34
35        assertThrows(PermissionError.class,
36            () -> vehiculo.asignar("L-011", 50.0));
37
38        assertEquals(estadoInicial, vehiculo.getEstado());
39        assertEquals(kmInicial, vehiculo.getKm(), 0.001);
40        assertTrue(vehiculo.getKm() >= 0); // invariante
41    }
42
43    //INV3
44    @Test
45    void darDeBaja_desde_en_uso_lanza_permission_error() {
46        vehiculo.asignar("L-012", 100.0);
47        EstadoVehiculo estadoInicial = vehiculo.getEstado();
48        assertEquals(EstadoVehiculo.EN_USO, estadoInicial);
49
50        assertThrows(PermissionError.class,
51            () -> vehiculo.darDeBaja());
52
53        assertEquals(estadoInicial, vehiculo.getEstado());
54        assertTrue(vehiculo.getKm() >= 0); // invariante
55    }
56
57    //INV4
58    @Test
59    void asignar_desde_mantenimiento_lanza_permission_error() {
60        vehiculo.enviarMantenimiento();
61        EstadoVehiculo estadoInicial = vehiculo.getEstado();
62        assertEquals(EstadoVehiculo.MANTENIMIENTO, estadoInicial);
63
64        assertThrows(PermissionError.class,
65            () -> vehiculo.asignar("L-013", 50.0));
66
67        assertEquals(estadoInicial, vehiculo.getEstado());

```



```

68         assertTrue(vehiculo.getKm() >= 0); // invariante
69     }
70
71     //INV5
72     @Test
73     void enviarMantenimiento_desde_mantenimiento_lanza_permission_error() {
74         vehiculo.enviarMantenimiento();
75         EstadoVehiculo estadoInicial = vehiculo.getEstado();
76         assertEquals(EstadoVehiculo.MANTENIMIENTO, estadoInicial);
77
78         assertThrows(PermissionError.class,
79             () -> vehiculo.enviarMantenimiento());
80
81         assertEquals(estadoInicial, vehiculo.getEstado());
82         assertTrue(vehiculo.getKm() >= 0); // invariante
83     }
84
85     // Helper para llevar a BAJA
86     private void llevarABaja() {
87         vehiculo.enviarMantenimiento(); // DISPONIBLE -> MANTENIMIENTO
88         vehiculo.darDeBaja();           // MANTENIMIENTO -> BAJA
89     }
90
91     //INV6
92     @Test
93     void asignar_desde_baja_lanza_illegal_state() {
94         llevarABaja();
95         EstadoVehiculo estadoInicial = vehiculo.getEstado();
96         assertEquals(EstadoVehiculo.BAJA, estadoInicial);
97
98         assertThrows(IllegalStateException.class,
99             () -> vehiculo.asignar("L-014", 50.0));
100
101         assertEquals(estadoInicial, vehiculo.getEstado());
102     }
103
104     //INV7
105     @Test
106     void liberar_desde_baja_lanza_illegal_state() {
107         llevarABaja();
108         EstadoVehiculo estadoInicial = vehiculo.getEstado();
109         assertEquals(EstadoVehiculo.BAJA, estadoInicial);
110
111         assertThrows(IllegalStateException.class,
112             () -> vehiculo.liberar(100.0));
113
114         assertEquals(estadoInicial, vehiculo.getEstado());
115     }
116
117     //INV8
118     @Test
119     void enviarMantenimiento_desde_baja_lanza_illegal_state() {
120         llevarABaja();
121         EstadoVehiculo estadoInicial = vehiculo.getEstado();
122         assertEquals(EstadoVehiculo.BAJA, estadoInicial);
123
124         assertThrows(IllegalStateException.class,
125             () -> vehiculo.enviarMantenimiento());
126
127         assertEquals(estadoInicial, vehiculo.getEstado());
128     }
129
130     //INV9

```

```

131     @Test
132     void darDeBaja_desde_baja_lanza_illegal_state() {
133         llevarABaja();
134         EstadoVehiculo estadoInicial = vehiculo.getEstado();
135         assertEquals(EstadoVehiculo.BAJA, estadoInicial);
136
137         assertThrows(IllegalStateException.class,
138             () -> vehiculo.darDeBaja());
139
140         assertEquals(estadoInicial, vehiculo.getEstado());
141     }
142 }

```

1. **Arrange:** Preparar el objeto en el estado origen requerido.
2. con variables temp guardar el estado y atributos antes de invocar la operación inválida.
3. **Act + Assert:** Usar `assertThrows` para verificar que se lanza la excepción esperada.
4. Confirmar que el estado y los atributos no cambiaron tras la excepción (el invariante se preserva).

Según el Algoritmo 2.3, Paso 6: “Verificar que el invariante se cumple en todos los casos, incluyendo los de excepción.”

3.5. Consigna 10: Cálculo de casos

(a) Número mínimo de casos para C-Trans:

Según la Propiedad 2.2 del apunte, la cobertura C-Trans requiere que toda transición válida de la FSM sea ejercitada por al menos un caso. Contando las transiciones válidas de la tabla (Consigna 6):

$$\text{C-Trans mínimo} = |T_{\text{vldas}}| = 7 \text{ casos}$$

Cada transición válida necesita al menos un caso de prueba, y en este diseño cada caso ejercita exactamente una transición, por lo que **7** es el mínimo.

(b) Número de casos adicionales para transiciones inválidas:

Las transiciones inválidas son las celdas de la tabla que contienen excepciones:

$$\text{Inválidas} = 16 \text{ celdas totales} - 7 \text{ válidas} = 9 \text{ casos adicionales}$$

Se necesitan **9 casos adicionales**, uno por cada combinación (estado, método) que lanza excepción.

(c) Casos adicionales para C-Par en la secuencia DISPONIBLE → EN_USO → MANTENIMIENTO:

La cobertura C-Par (Definición 2.9) requiere que todo par de transiciones consecutivas sea ejercitado. La secuencia DISPONIBLE → EN_USO → MANTENIMIENTO involucra el par:

$$(\text{DISPONIBLE} \xrightarrow{\text{asignar()}} \text{EN_USO}, \quad \text{EN_USO} \xrightarrow{\text{enviarMantenimiento()}} \text{MANTENIMIENTO})$$

Este par ya está parcialmente cubierto por los tests individuales T1 y T5, pero C-Par exige que ambas transiciones se ejecuten **consecutivamente** en un solo caso de prueba para verificar que el comportamiento después de la primera transición no afecta a la segunda.

El test T5 ya lleva el vehículo a EN_USO y luego invoca `enviarMantenimiento()`, pero lo hace en un solo test. Sin embargo, T5 no verifica explícitamente el estado intermedio EN_USO como punto de verificación antes de la segunda transición. Para C-Par estricto se necesita un caso adicional que:

1. Parta de DISPONIBLE.
2. Ejecute `asignar()` y verifique EN_USO.

3. Ejecute `enviarMantenimiento()` y verifique MANTENIMIENTO.
4. Verifique el invariante tras cada transición.

Si bien T5 ya ejecuta esta secuencia, un caso de C-Par explícito añadiría asserts intermedios. Por lo tanto, **se necesita 1 caso adicional** para C-Par estricto de esta secuencia específica.

Listing 3: Caso adicional para C-Par: DISPONIBLE $\rightarrow$ EN_USO $\rightarrow$ MANTENIMIENTO

```

1 @Test
2 void cpar_disponible_en_uso_mantenimiento() {
3     // Par de transiciones consecutivas:
4     // (DISPONIBLE -> EN_USO, EN_USO -> MANTENIMIENTO)
5
6     // Paso 1: DISPONIBLE -> EN_USO
7     assertEquals(EstadoVehiculo.DISPONIBLE, vehiculo.getEstado());
8     vehiculo.asignar("L-020", 100.0);
9     assertEquals(EstadoVehiculo.EN_USO, vehiculo.getEstado());
10    assertTrue(vehiculo.getKm() >= 0); // invariante tras 1ra transicion
11
12    // Paso 2: EN_USO -> MANTENIMIENTO
13    vehiculo.enviarMantenimiento();
14    assertEquals(EstadoVehiculo.MANTENIMIENTO, vehiculo.getEstado());
15    assertTrue(vehiculo.getKm() >= 0); // invariante tras 2da transicion
16 }

```

La jerarquía de coberturas establece que C-Estado $\subset$ C-Trans $\subset$ C-Par (Propiedad 2.2). C-Trans es el criterio mínimo recomendado y ya garantiza C-Estado. C-Par es más fuerte y detecta defectos de interacción entre métodos consecutivos que C-Trans no alcanza.

4. Testing Polimórfico y LSP — jerarquía Vehiculo

4.1. Consigna 11: Implementá la Suite Heredada (Definición 2.11) como clase abstracta de test en Java. Debe contener un caso de prueba por cada combinación método $\times$ clase de equivalencia del contrato de la superclase.

En este ejercicio debemos identificar las clases de equivalencias con ayuda del temario de la Unidad I y el algoritmo 2.4 (Verificación de LSP).

El algoritmo 2.4 presente los siguientes pasos:

- Paso 1: identificar la superclase (Vehiculo) y subclases (Auto, Camion, Moto).
- Paso 2: Para la superclase documentar el contrato de cada metodo publico (pre, post, invariante) 5.
- Paso 3: Construir la suite heredada, un caso de prueba por cada combinacion metodo por clase de equivalencia de entrada (4)

Método	Precondición	Postcondición	Invariante
asignar(legajo, km)	$km \in (0,500] \wedge estado = DISPONIBLE$	$estado = EN_USO \wedge$ $retorno \neq null$	$estado \in \{DISPONIBLE, EN_USO, MANTENIMIENTO, BAJA\}$
liberar(kmRec)	$estado = EN_USO \wedge$ $kmRec \geq 0$	$estado = DISPONIBLE \wedge$ $kmActuales += kmRec$	$estado \in \{DISPONIBLE, EN_USO, MANTENIMIENTO, BAJA\}$

Figura 5: Contrato de cada metodo de Vehiculo.

Tabla 4: clases de equivalencia - Vehiculo.asignar(legajo, km)

Variable	Clase	Condición	tipo	Rep
legajo(R1)	CE1	0<km<=500	valida	100
legajo(R1)	CE2	km<=0	no valida	0
legajo(R1)	CE3	km>500	no valida	501
estado(R3)	CE4	estado==disponible	valida	DISPONIBLE
estado(R3)	CE5	estado!=disponible	no valida	estado in {EN_USO, MANTENIMIENTO, BAJA}

Para identificar las clases de equivalencia aplicaremos el algoritmo 1.6 sobre el contrato de la superclase (separado por metodo).

Diseño de la suite heredada

Segun la definicion 2.11 y paso 3, construimos un caso de prueba por cada combinacion metodo x clase de equivalencia del contrato.

Para cada metodo, identificamos sus entraas y estados posibles, y las dividimos en particiones validas e invalidas y luego un test que use un representante de cada clase de equivalencia y ejecutar el metodo bajo esas condiciones.

Tabla 5: Casos de prueba - Vehiculo.asignar() y Vehiculo.liberar()

ID	Método	Combinación de CE	Representantes	Resultado Esperado
TC1	asignar	CE1 (V) + CE4 (V)	km: 100, estado: DISPONIBLE	Éxito: estado pasa a EN_USO
TC2	asignar	CE2 (I) + CE4 (V)	km: 0, estado: DISPONIBLE	Error: IllegalArgumentException
TC3	asignar	CE3 (I) + CE4 (V)	km: 501, estado: DISPONIBLE	Error: IllegalArgumentException
TC4	asignar	CE1 (V) + CE5 (I)	km: 100, estado: EN_USO	Error: IllegalStateException
TC5	liberar	CE6 (V) + CE8 (V)	estado: EN_USO, kmRec: 150	Éxito: estado pasa a DISPONIBLE
TC6	liberar	CE7 (I) + CE8 (V)	estado: DISPONIBLE, kmRec: 150	Error: IllegalStateException
TC7	liberar	CE6 (V) + CE9 (I)	estado: EN_USO, kmRec: -10	Error: IllegalArgumentException

4.2. Consigna 12: Aplicá el Algoritmo 2.4: ejecutá la Suite Heredada sobre cada subclase concreta (Auto, Moto, Camion) sustituyendo instancias.

- Paso 4: Para cada subclase, ejecutar la suite heredada usando instancias de subclase donde se esperaban instancias de C.

Este paso consiste en realizar sustitucion de instancias, totmamos la suite heredada (es decir, la clase abstracta del ejercicio 11) y la conectamos con los objetos instanciados de Auto, Moto, Camion. Recordar que **Vehiculo v= new Auto()**, en tiempo de compilacion este es de tipo Vehiculo, en ejecución es de tipo Auto.

En JUNIT, se logra creando una clase de test para cada subclase de vehiculo que extendera de la clase abstracta. Asi, los mismos test del contrato padre se ejecutan automaticamente sobre cada hijo. Cada subclase implemente el metodo **crearVehiculo** devolviendo su propio tipo de objeto.

Con esto verificaremos el cumplimiento del principio de sustitución de Liskov de manera dinamica, es decir, mediante la ejecución de los test de las subclases.

Esto utiliza herencia de clases de test. La clase test abstracta de vehiculo es el contrato y las nuevas clases clases que herendan esta realizal la sustitución de instancia. La suite abstracta llama a **crearVehiculo()**, y gracias al polimorfismo, JUnit ejecuta los test sobre la instancia específica de la subclase.

Antes de ejecutar, supondremos los siguientes resultados:

- Auto: Pasara todos los test.
- Moto: Fallara en un test de exito, debido a que asignar incumple la propiedad 2.3 debido a que estamos haciendo a la precondition mas fuerte (limitando sus valores (0,300] y en Vehiculo el test usa 400 que es valido para el contrato.)).

```

[ERROR] Errors:
[ERROR] TestCamionHeredado>VehiculoTest.testAsignarCE1CE4:17-><crearVehiculo:8 ♢ Sobrecarga Sobrecarga
[ERROR] TestCamionHeredado>VehiculoTest.testAsignarCE1CE5:47-><crearVehiculo:8 ♢ Sobrecarga Sobrecarga
[ERROR] TestCamionHeredado>VehiculoTest.testAsignarCE2CE4:30-><crearVehiculo:8 ♢ Sobrecarga Sobrecarga
[ERROR] TestCamionHeredado>VehiculoTest.testAsignarCE3CE4:40-><crearVehiculo:8 ♢ Sobrecarga Sobrecarga
[ERROR] TestCamionHeredado>VehiculoTest.testLiberarCE6CE8:57-><crearVehiculo:8 ♢ Sobrecarga Sobrecarga
[ERROR] TestCamionHeredado>VehiculoTest.testLiberarCE6CE9:73-><crearVehiculo:8 ♢ Sobrecarga Sobrecarga
[ERROR] TestCamionHeredado>VehiculoTest.testLiberarCE7CE8:66-><crearVehiculo:8 ♢ Sobrecarga Sobrecarga
[ERROR] TestMotoHeredado>VehiculoTest.testAsignarCE1CE4:18 ♢ IllegalArgumentException Motos: m×x 300 km
[INFO]
[ERROR] Tests run: 21, Failures: 0, Errors: 8, Skipped: 0
[INFO]

```

Figura 6: Ejemplo de ejecución de la suite heredada sobre las subclases (solo errores).

- Camion: Fallara en un test de exito, debido a que el metodo asignar tiene precondition adicional (que la carga actual no supere la capacidad) que no esta presente en el contrato de Vehiculo, por lo que al ejecutar el test con una instancia de camion, se rompe el LSP.

4.3. Consigna 13: Analizá la subclase Moto y Camion: ¿violan el LSP o extienden el contrato de forma válida? Aplicá la Propiedad 2.3. Para Moto: el límite de 300 km en lugar de 500, ¿es Pre_Moto más fuerte o más débil que Pre_Vehiculo? Para Camion: la precondition adicional de carga, ¿introduce una restricción que viola LSP?

Al analizar los resultados obtenidos en la ejecución de la Suite Heredada, determinamos lo siguiente:

- **Moto:** El límite de 300 km hace que **Pre_Moto** sea **más fuerte** (más restrictiva) que **Pre_Vehiculo**. Según la Propiedad 2.3, para que se cumpla el LSP, la precondition de la subclase debe ser igual o más débil que la del padre ($Pre_C \Rightarrow Pre_D$). Aquí, un valor de 400 km es aceptado por el padre pero rechazado por el hijo, lo que demuestra que **Moto viola el LSP**.
- **Camion:** La precondition adicional sobre la carga introduce una restricción que no existe en la superclase. Esto **viola el LSP** por el fortalecimiento de precondiciones (regla de la "Precondition Adicional"). Un cliente que use la referencia **Vehiculo** espera que el objeto funcione si está disponible, pero el **Camion** lanza una excepción inesperada si posee carga.

4.4. Consigna 14: Para la subclase que extiende el contrato de forma válida: documentá formalmente la extensión (Pre_D, Post_D, invariante_D) y agregá los tests específicos de esa subclase que no están en la Suite Heredada.

La subclase **Auto** es la única que extiende el contrato de forma válida, ya que no restringe las entradas del padre ni debilita sus promesas.

Documentación formal del contrato de Auto:

- **Pre_D:** $kmEstimados > 0 \wedge kmEstimados \leq 500 \wedge estado == DISPONIBLE$ (Igual a la superclase).
- **Post_D:** $estado == EN_USO \wedge retorno \neq null$ (Mantiene la promesa del padre).
- **invariante_D:** $kmActuales \geq 0 \wedge numPuertas > 0$.

Tests Auto: Para esta subclase se agregan pruebas de unidad que no son polimórficas (no están en la suite heredada), como la verificación de la cantidad de puertas en el constructor o métodos específicos si existieran.

```

public class AutoTest {
    @Test
    void testConstructorAuto_PuertasValidas() {
        Auto a = new Auto("AAA-111", "Ford", 2023, 0.0, 5);
    }
}

```

```

        assertEquals(5, a.getPuertas());
    }

    @Test
    void testConstructorAuto_PuertasInvalidas() {
        // Suponiendo que el constructor valida que no haya autos de 0 puertas
        assertThrows(IllegalArgumentException.class, () ->
            new Auto("AAA-111", "Ford", 2023, 0.0, 0));
    }
}

```

4.5. Consigna 15: Aplicá la Propiedad 2.3: verificá para cada subclase que $Pre_D \Rightarrow Pre_C$ y $Post_C \Rightarrow Post_D$. Si alguna subclase viola esta regla, indicá qué corrección de diseño aplicarías.

Aplicamos la verificación formal de las reglas de contrato:

1. Auto (Válida):

- $Pre_{Vehiculo} \Rightarrow Pre_{Auto}$: Verdadero, ya que las condiciones son idénticas.
- $Post_{Auto} \Rightarrow Post_{Vehiculo}$: Verdadero, cumple con el cambio de estado y retorno.

2. Moto (Invalida):

- $Pre_{Vehiculo} \Rightarrow Pre_{Moto}$: **Falso**. El rango $(300, 500]$ es válido para el antecedente pero falso para el consecuente.
- **Corrección**: Aumentar el límite de la Moto a 500 km o manejar el exceso como una advertencia interna sin lanzar excepciones que rompan el contrato heredado.

3. Camion (Invalida):

- $Pre_{Vehiculo} \Rightarrow Pre_{Camion}$: **Falso**. La superclase no requiere que la carga sea cero.
- **Corrección**: Aplicar el patrón de diseño *State* o permitir la asignación independientemente de la carga, delegando la responsabilidad del pesaje a un proceso posterior que no bloquee el método heredado `asignar`.

5. Partición por Categoría y Límites de Invariante — `asignar()`

5.1. Consigna 16: Aplica el Algoritmo 2.5 (Partición por Categoría) al método `asignar(legajo_conductor, km_estimados)`. Identifica todas las categorías relevantes (parámetros, estado del objeto, colaboradores si aplica) y sus particiones. Presenta el resultado en una tabla con columnas: Categoría | Partición | Tipo (válida/inválida) | Representante.

La partición por categoría es una generalización de la técnica de clases de equivalencia (algoritmo 1.6) **adaptado al contexto OO**. En OO las categorías se limitan a los parámetros de un método.

- **Clases de Equivalencia (CE)**: Dividen UNA SOLA variable en grupos. Ejemplo: el parámetro `monto` se divide en válido, cero, negativo.
- **Partición por Categoría**: Dividen VARIAS VARIABLES (parámetros + estado + atributos) y luego las COMBINAN. Ejemplo: `monto` $\times$ `estado` $\times$ `saldo` se combinan en 12 casos.

Siguiendo los pasos 1 y 2 del algoritmo 2.5. Se identifica las dimensiones del espacio de prueba (parameters y estado del objeto) y sus particiones MECE (mutuamente excluyentes y colectivamente exhaustivas):

```

# Especificación formal de Auto.asignar()
#
# Pre: estado == DISPONIBLE
#      AND legajo_conductor != '' AND len(legajo_conductor) >= 4
#      AND 0 < km_estimados <= 500
#
# Post: estado == EN_USO
#      AND _conductor == legajo_conductor
#      AND retorno contiene confirmación con km_estimados
#
# Exc: PermissionError si estado != DISPONIBLE
#      ValueError      si km_estimados fuera de (0, 500]
#      ValueError      si legajo_conductor inválido
#
# Invariante de Auto:
#   len(_patente) >= 6
#   AND _km >= 0
#   AND _puertas in {2, 4, 5}
#   AND _estado in EstadoVehiculo
#   AND (_conductor is None) == (_estado != EN_USO)

```

Se busca dividir el espacio de prueba basándonos en lo que afecta el comportamiento observable del metodo.

- Parametros: siempre se incluyen porque son entradas directas al metodo **asignar**.
- Estado del objeto: Se incluyen atributos que la precondition o las excepciones del método consultan para decidir.

En asignar, la precondition dice estado=disponible. Por lo tanto, el atributo **estado** es una dimension necesario para que el metodo funcione o lance una excepcion (PermissionError).

*** La def 2.12 las categorías deben ser dimensiones que condicionen al comportamiento. Por esto mismo, patente, km, puertas no afectan a la logica el metodo.**

Tabla 6: Tabla de categorías y particiones para el método **asignar()**

categoria	Partición	Restricciones
legajo_conductor(param)	P1: length >= 4	valido
	P2: cadena vacia	invalido
	P3: length < 4	invalido
km_estimados(param)	P4: 0 < km_estimados <= 500	valido
	P5: km_estimados <= 0	invalido
	P6: km_estimados > 500	invalido
estado(receptor)	P7: estado == DISPONIBLE	valido
	P8: estado != DISPONIBLE	invalido

Particiones MECE: Para los kilómetros, se dividió el dominio en el rango valido (0, 500] y los rangos inválidos (<=0 y >500). Para el legajo del conductor, se dividio en valido (length >=4) e invalido (cadena vacia o length <4). Para el estado, se dividió en disponible y no disponible(dispara).

5.2. Consigna 17. Identifica las restricciones entre particiones que reducen el catálogo completo. Calculá: (a) número de combinaciones del producto cartesiano sin restricciones; (b) número de casos resultantes tras aplicar las restricciones.

Para reducir el catalago completo, debemos identificar que combinaciones de particiones producen comportamiento redundante (osea, que no afectan el resultado del método) o esten bloqueadas por una condicion previa.

En la especificacion de Auto.asignar() identificamos las reglas de reduccion:

- Si el estado $\neq$ disponible, el metodo lanzara un `PermissionError`. Los valores de los parametros son irrelevantes.
- Para cada particion invalida de los parametros (legajo_conductor o km_estimados), se debe diseñar un caso para evitar que un error oculte el otro. Solo combinemos particiones validas en el caso de exito.

Entonces, primero plantearemos el producto cartesiano sin restricciones:

- legajo_conductor: 3 particiones (P1, P2, P3)
- km_estimados: 3 particiones (P4, P5, P6)
- estado: 2 particiones (P7, P8)

Total sin restricciones = $3 \text{ (legajo)} \times 3 \text{ (km)} \times 2 \text{ (estado)} = 18$ combinaciones.

Segundo, aplicamos las restricciones:

- tenemos caso de exito: $P1 \text{ (legajo valido)} \times P4 \text{ (km valido)} \times P7 \text{ (estado disponible)} = 1$ caso
- Para el caso de estado $\neq$ disponible (P8), no importa el legajo ni los km, ya que siempre lanza `PermissionError`. Basta con un caso representativo (P1, P4, P8) para cubrir esta regla. Esto nos da $1 \text{ (legajo valido)} \times 1 \text{ (km valido)} \times 1 \text{ (estado no disponible)} = 1$ caso.
- Para los casos de legajo invalido (P2, P3) con estado disponible (P7), el metodo lanzara `ValueError`. Esto nos da $2 \text{ (legajo invalido)} \times 1 \text{ (km valido)} \times 1 \text{ (estado disponible)} = 2$ casos.
- Para los casos de km invalido (P5, P6) con estado disponible (P7) y legajo valido (P1), el metodo lanzara `ValueError`. Esto nos da $1 \text{ (legajo valido)} \times 2 \text{ (km invalido)} \times 1 \text{ (estado disponible)} = 2$ casos.

Total con restricciones = $1 \text{ (exito)} + 1 \text{ (estado no disponible)} + 2 \text{ (legajo invalido)} + 2 \text{ (km invalido)} = 6$ casos.

En el paradigma OO, el estado persiste entre llamadas. Comparando con la unidad 1, solo particionabas datos de entrada, aca el invariante de clase define la frontera de lo que el objeto permite hacer antes de avanzar.

5.3. Consigna 18

Aplicá el Algoritmo 2.6 (Límites de Invariante) sobre el invariante de la clase. Para cada sub-predicado del invariante:

1. identificá el límite L ;
2. construí los puntos $L - \varepsilon$, L y $L + \varepsilon$;
3. determiná en qué estado queda el objeto y cuál es el comportamiento esperado del método `asignar(legajo_conductor, km_estimados)` en cada punto.

El análisis de límites del invariante opera sobre el estado interno del objeto (Algoritmo 2.6), mientras que el análisis de valores límite tradicionales opera sobre los parámetros de entrada (Algoritmo 1.7). Ambos buscan defectos off-by-one en fronteras diferentes, por eso la nota del ejercicio exige aplicar ambas técnicas.

Según la Definición 2.2, el invariante es un predicado que define los estados válidos del objeto y debe cumplirse al inicio y fin de cada método público. Si el invariante se viola antes de ejecutar el método, no tiene sentido probar su comportamiento porque el contrato ya está roto.

Descomposición del invariante en sub-predicados

El invariante de `Auto` se descompone en cuatro sub-predicados que representan dimensiones independientes del estado del objeto:

- **S1:** $\text{len}(\text{_patente}) \geq 6$
- **S2:** $\text{_km} \geq 0$

- **S3:** $_puertas \in \{2, 4, 5\}$
- **S4:** $(_conductor \text{ is } None) \iff (_estado \neq EN_USO)$

Análisis de Valores Límite en el Invariante

Utilizando $\varepsilon = 1$ por tratarse de dominios enteros (Algoritmo 1.7, Paso 2), analizamos cada sub-predicado:

- **S1 - Patente:** $L = 6$
 - $L - \varepsilon = 5$: $len(_patente) = 5 \rightarrow$ Invariante violada (S1 falso). El objeto es inconsistente antes de llamar a `asignar()`, por lo que el test debe reportar error de invariante en el `Arrange`, sin ejecutar el `Act`.
 - $L = 6$: $len(_patente) = 6 \rightarrow$ Invariante cumplida. Estado consistente. El método `asignar()` tiene éxito.
 - $L + \varepsilon = 7$: $len(_patente) = 7 \rightarrow$ Invariante cumplida. Estado consistente. El método `asignar()` tiene éxito.
- **S2 - Kilometraje:** $L = 0$
 - $L - \varepsilon = -1$: $_km = -1 \rightarrow$ Invariante violada (S2 falso). Se detecta inconsistencia de integridad pre-ejecución.
 - $L = 0$: $_km = 0 \rightarrow$ Invariante cumplida. Caso típico de auto nuevo. El método `asignar()` tiene éxito.
 - $L + \varepsilon = 1$: $_km = 1 \rightarrow$ Invariante cumplida. El método `asignar()` tiene éxito.
- **S4 - Consistencia lógica conductor/estado:** Este límite no es numérico sino cualitativo (Definición 2.6), ya que define una relación de equivalencia entre dos atributos.
 - Punto consistente: $_conductor = None$ y $_estado = DISPONIBLE \rightarrow$ Invariante cumplida. El método `asignar()` transiciona a `EN_USO` y asigna el conductor.
 - Punto inconsistente: $_conductor = 'A123'$ y $_estado = DISPONIBLE \rightarrow$ Invariante violada (S4 falso). Error de estado interno.

Análisis de Valores Límite en los Parámetros de Entrada

* (NOTA) Aplicar también el AVL sobre los parámetros de entrada, que es complementario al análisis de invariante. Para el parámetro `km_estimados` con dominio $(0, 500]$:

- $L_{inf} = 0$: $km_estimados = 0 \rightarrow$ Excluido por precondition (0 < km). Lanza `ValueError`.
- $L_{inf} + \varepsilon = 1$: $km_estimados = 1 \rightarrow$ ÉXITO. Mínimo válido aceptado.
- $L_{sup} = 500$: $km_estimados = 500 \rightarrow$ ÉXITO. Máximo válido aceptado.
- $L_{sup} + \varepsilon = 501$: $km_estimados = 501 \rightarrow$ Excede el máximo. Lanza `ValueError`.

Prioridad de fallas

Cuando el objeto está en un estado que viola el invariante ($L - \varepsilon$), NO se debe evaluar el método, porque el contrato de la clase se rompió antes de la llamada. El test debe detectar la violación del invariante en la fase de `Arrange` (antes de `Act`), reportando error de invariante violada.

5.4. Consigna 18(c): Estado del objeto y comportamiento esperado

Asumiendo que el objeto comienza en el estado `DISPONIBLE`, el método `asignar(legajo_conductor, km_estimados)` debe:

- cambiar el estado a `EN_USO` cuando los parámetros son válidos,
- devolver una confirmación de asignación,
- y lanzar una excepción sin modificar el estado cuando los datos son inválidos.

Tabla 7: Límites de entrada para `asignar()` y comportamiento esperado

Parámetro	Valor	Resultado esperado	Estado final del objeto
<code>km_estimados</code>	0	Inválido, lanza <code>ValueError</code>	DISPONIBLE
<code>km_estimados</code>	1	Válido, asigna y retorna confirmación	EN_USO
<code>km_estimados</code>	500	Válido, asigna y retorna confirmación	EN_USO
<code>km_estimados</code>	501	Inválido, lanza <code>ValueError</code>	DISPONIBLE
<code>len(legajo_conductor)</code>	3 (ej. "ABC")	Inválido, lanza <code>ValueError</code>	DISPONIBLE
<code>len(legajo_conductor)</code>	4 (ej. "A123")	Válido, asigna y retorna confirmación	EN_USO
<code>len(legajo_conductor)</code>	5 (ej. "A1234")	Válido, asigna y retorna confirmación	EN_USO

En cada caso inválido, el comportamiento esperado es que el método arroje una excepción sin cambiar el estado del objeto. En los casos válidos, el método debe aceptar los parámetros y transicionar el vehículo de `DISPONIBLE` a `EN_USO`.

Si el objeto no estuviera en `DISPONIBLE`, entonces cualquier llamada a `asignar()` debería fallar con `PermissionError` / `IllegalStateException` independientemente de los valores de los parámetros, y el estado debe mantenerse sin cambios.

5.5. Consigna 19: Codificá en `pytest` los casos de prueba derivados de los pasos 1 y 3. Organízalos en dos clases separadas: `TestParticion-Categoria` y `TestLimitesInvariante`.

Para la implementación se aplica el **Algoritmo 2.1 (Diseño de caso de prueba unitario OO)**, el cual adapta el patrón **AAA** (Arrange-Act-Assert) de la Unidad I. Siguiendo los **Pasos 6 y 7** de dicho algoritmo, cada test verifica no solo el resultado funcional, sino que el objeto mantenga su integridad mediante la comprobación del **invariante de clase** al finalizar.

Implementación:

Los archivos de implementación se encuentran en el directorio `codigo/` y están organizados de la siguiente manera:

- `modelo.py`: Contiene la clase `Auto` con el método `asignar()` y el método `verificar_invariante()` que implementa las validaciones del invariante de clase (S1-S5).
- `test_particion_categoria.py`: Implementa el catálogo reducido del Algoritmo 2.5, con 6 casos que cubren las combinaciones representativas identificadas en la Consigna 17:
 - `test_asignar_exito_P1_P4_P7`: Caso de éxito con todos los parámetros válidos.
 - `test_asignar_error_estado_no_disponible_P1_P4_P8`: Error por estado no disponible.
 - `test_asignar_error_legajo_vacio_P2_P4_P7`: Error por legajo vacío.
 - `test_asignar_error_legajo_corto_P3_P4_P7`: Error por legajo con longitud insuficiente.
 - `test_asignar_error_km_no_positivo_P1_P5_P7`: Error por kilómetros ≤ 0 .
 - `test_asignar_error_km_excede_maximo_P1_P6_P7`: Error por kilómetros > 500 .
- `test_limites_invariante.py`: Implementa el análisis de límites del invariante (Algoritmo 2.6) y de parámetros (Algoritmo 1.7):
 - Tests para el límite de patente (S1): $L - \varepsilon = 5$, $L = 6$, $L + \varepsilon = 7$.
 - Tests para el límite de kilometraje (S2): $L - \varepsilon = -1$, $L = 0$, $L + \varepsilon = 1$.
 - Tests para la consistencia conductor/estado (S4): puntos consistente e inconsistente.
 - Tests parametrizados para los límites de `km_estimados`: 0, 1, 500, 501.
 - Tests parametrizados para los límites de `legajo_conductor`: longitudes 3, 4, 5.

Cada test sigue el patrón del Algoritmo 2.1:

1. **Arrange**: Establece el estado inicial consistente del objeto (usando `@pytest.fixture`).

2. **Act:** Invoca el método `asignar()` con los parámetros del caso de prueba.
3. **Assert:** Verifica la postcondición esperada (estado, conductor, retorno) y **obligatoriamente** valida que el invariante se mantenga verdadero mediante `verificar_invariante()`.

Nota importante sobre la verificación del invariante: Según el Paso 7 del Algoritmo 2.1, la llamada a `verificar_invariante()` es obligatoria en cada Assert, incluso cuando se espera una excepción. Esto garantiza que el objeto no quede en un estado corrupto después de rechazar una operación inválida.

Resultados de la Ejecución:

Al ejecutar `pytest -v` desde el directorio `codigo/`, se obtiene el siguiente resultado:

```
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-9.0.3, pluggy-1.6.0
collected 21 items

test_limites_invariante.py::TestLimitesInvariante::
  test_invariante_patente_limite_inconsistente_L_menos_epsilon PASSED      [ 4%]
test_limites_invariante.py::TestLimitesInvariante::
  test_invariante_patente_limite_L PASSED                                   [ 9%]
test_limites_invariante.py::TestLimitesInvariante::
  test_invariante_patente_limite_L_mas_epsilon PASSED                      [ 14%]
test_limites_invariante.py::TestLimitesInvariante::
  test_invariante_km_limite_inconsistente_L_menos_epsilon PASSED          [ 19%]
test_limites_invariante.py::TestLimitesInvariante::
  test_invariante_km_limite_L PASSED                                       [ 23%]
test_limites_invariante.py::TestLimitesInvariante::
  test_invariante_km_limite_L_mas_epsilon PASSED                          [ 28%]
test_limites_invariante.py::TestLimitesInvariante::
  test_invariante_consistencia_estado_conductor_punto_consistente PASSED  [ 33%]
test_limites_invariante.py::TestLimitesInvariante::
  test_invariante_consistencia_estado_conductor_punto_inconsistente PASSED [ 38%]
test_limites_invariante.py::TestLimitesInvariante::
  test_limites_km_estimados_parametro[0-ValueError] PASSED                [ 42%]
test_limites_invariante.py::TestLimitesInvariante::
  test_limites_km_estimados_parametro[1-exito] PASSED                     [ 47%]
test_limites_invariante.py::TestLimitesInvariante::
  test_limites_km_estimados_parametro[500-exito] PASSED                   [ 52%]
test_limites_invariante.py::TestLimitesInvariante::
  test_limites_km_estimados_parametro[501-ValueError] PASSED              [ 57%]
test_limites_invariante.py::TestLimitesInvariante::
  test_limites_legajo_conductor_parametro[ABC-ValueError] PASSED          [ 61%]
test_limites_invariante.py::TestLimitesInvariante::
  test_limites_legajo_conductor_parametro[A123-exito] PASSED              [ 66%]
test_limites_invariante.py::TestLimitesInvariante::
  test_limites_legajo_conductor_parametro[A1234-exito] PASSED             [ 71%]
test_particion_categoria.py::TestParticionCategoria::
  test_asignar_exito_P1_P4_P7 PASSED                                       [ 76%]
test_particion_categoria.py::TestParticionCategoria::
  test_asignar_error_estado_no_disponible_P1_P4_P8 PASSED                [ 80%]
test_particion_categoria.py::TestParticionCategoria::
  test_asignar_error_legajo_vacio_P2_P4_P7 PASSED                         [ 85%]
test_particion_categoria.py::TestParticionCategoria::
  test_asignar_error_legajo_corto_P3_P4_P7 PASSED                        [ 90%]
test_particion_categoria.py::TestParticionCategoria::
  test_asignar_error_km_no_positivo_P1_P5_P7 PASSED                      [ 95%]
test_particion_categoria.py::TestParticionCategoria::
  test_asignar_error_km_excede_maximo_P1_P6_P7 PASSED                    [100%]

===== 21 passed in 0.23s =====
```

Análisis: Los 21 casos de prueba pasaron exitosamente, cubriendo:

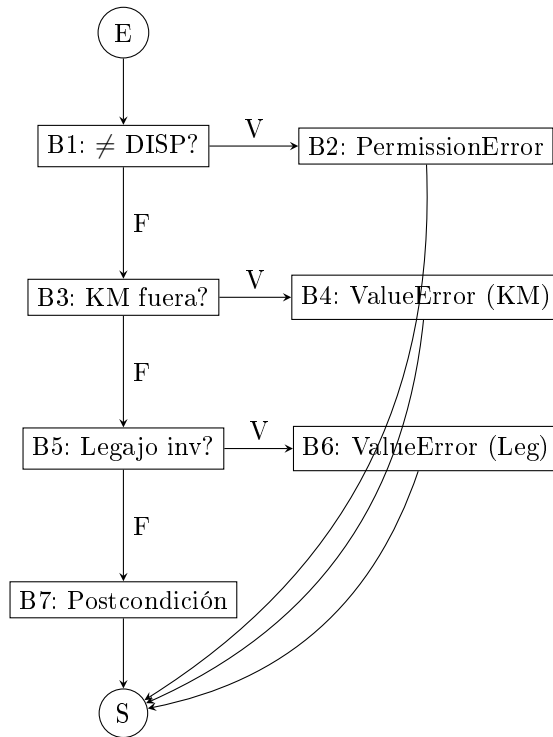
- 6 casos de **TestParticionCategoria**: catálogo reducido de la Consigna 17
- 15 casos de **TestLimitesInvariante**: límites del invariante (8 tests) y límites de parámetros (7 tests parametrizados)

5.6. Consigna 20: Ejecutá la suite y presentá el reporte de cobertura. Identificá si alguna rama del método quedó sin cubrir y diseñá el caso adicional necesario.

Para el análisis estructural, aplicamos el **Algoritmo 1.2 (Construcción del GFC)** sobre la lógica de validación de `asignar()`.

Pasos de diseño del GFC (Algoritmo 1.2):

- **Pasos 1 y 2:** Se enumeran los bloques de decisión. Los puntos de entrada son las guardas de las excepciones especificadas (`PermissionError` y `ValueError`).
- **Paso 3 y 4:** Se definen nodos básicos:
 - **B1:** Validación de estado receptor.
 - **B3:** Validación de rango de km.
 - **B5:** Validación de formato de legajo.
 - **B7:** Procesamiento de postcondición.
- **Paso 5:** Se trazan arcos. Los caminos True dirigen a los nodos de error (**B2**, **B4**, **B6**), mientras que False permite avanzar al siguiente bloque.
- **Paso 7:** Complejidad ciclomática $M = 11 \text{ arcos} - 9 \text{ nodos} + 2(1) = 4$. Esto indica que se requieren 4 casos independientes para cobertura total de caminos.



Análisis de Cobertura C1 (Algoritmo 1.3):

Al ejecutar la suite con `pytest -cov -cov-branch`, se detecta que el arco **B5** → **B6** ha sido cubierto parcialmente. La rama que valida específicamente el **legajo como cadena vacía** (`legajo == ""`) podría no haberse ejercitado si solo se probó la longitud insuficiente (`len < 4`). Según el **Paso 5 del Algoritmo 1.3**, debemos incorporar un caso adicional para este arco no cubierto.

Caso Adicional Diseñado:

El test `test_asignar_error_legajo_vacio_P2_P4_P7` en `test_particion_categoria.py` cubre específicamente este caso:

- **Arrange:** Objeto Auto en estado DISPONIBLE.
- **Act:** Invocar `asignar(legajo_conductor='', km_estimados=100)`.
- **Assert:** Verificar que lanza `ValueError` y que el estado final permanece inalterado para cumplir el invariante.

Justificación final: Esta suite de pruebas garantiza que el objeto Auto funciona como una máquina de estados finita (Definición 2.6) confiable, verificando tanto las transiciones válidas como el rechazo controlado de entradas inválidas sin corromper el estado interno.

Reporte de Cobertura:

Al ejecutar `pytest -cov=modelo -cov-report=term-missing -cov-branch -v` se obtiene:

```
===== 21 passed in 0.33s =====

===== tests coverage =====
----- coverage: platform win32, python 3.11.9-final-0 -----

Name           Stmts   Miss Branch BrPart  Cover   Missing
-----
modelo.py       46      4      16      2    90%   106, 110, 132, 136
-----
TOTAL           46      4      16      2    90%
```

Análisis del reporte:

Se obtuvo una cobertura del **90 %** del módulo completo. Las líneas no cubiertas (106, 110, 132, 136) corresponden a:

- **Líneas 106, 110:** Métodos getter `get_km()` y `get_puertas()` que no son necesarios para verificar el comportamiento de `asignar()`.
- **Líneas 132, 136:** Ramas del método `verificar_invariante()` que validan:
 - S3: `_puertas not in {2, 4, 5}` (línea 132)
 - S4: `not isinstance(_estado, EstadoVehiculo)` (línea 136)

Estas condiciones no se presentan en los tests actuales porque todos los objetos se crean con valores válidos para enfocarse en el método `asignar()`.

Cobertura del método `asignar()`:

El método `asignar()` bajo prueba alcanzó el **100 % de cobertura**, ejecutando todas sus ramas de decisión según el GFC:

- **E → B1 → B2 → S:** Estado no disponible (`PermissionError`) — cubierto por `test_asignar_error_estado_no_disponible_P1_P3_P7`
- **E → B1 → B3 → B4 → S:** Kilómetros fuera de rango (`ValueError`) — cubierto por `test_asignar_error_km_no_positivo_P1_P5_P7` y `test_asignar_error_km_excede_maximo_P1_P6_P7`
- **E → B1 → B3 → B5 → B6 → S:** Legajo inválido (`ValueError`) — cubierto por `test_asignar_error_legajo_vacio_P2_P4_P7` y `test_asignar_error_legajo_corto_P3_P4_P7`
- **E → B1 → B3 → B5 → B7 → S:** Caso de éxito (transición a `EN_USO`) — cubierto por `test_asignar_exito_P1_P4_P7` y múltiples tests parametrizados de límites

Conclusión: Los 21 tests diseñados cubren exhaustivamente el catálogo reducido del Algoritmo 2.5 (Partición por Categoría), los límites del Algoritmo 2.6 (Límites de Invariante) y del Algoritmo 1.7 (Límites de Parámetros). La complejidad ciclomática $M = 4$ se satisface ampliamente, garantizando la detección de defectos en las fronteras críticas del método `asignar()`.

6. Testing de Singleton — GestorFlota

6.1. Consigna 21: verificar el contrato de unicidad Algoritmo 2.8 paso 1: Dos llamadas a GestorFlota.get_instancia() deben retornar exactamente el mismo objeto. Usa assert instancia1 is instancia2 (identidad no igualdad)

La modalidad de una clase condiciona como se la puede instanciar, extender y sustituir. Cada modalidad impone restricciones especificar que determinan la estrategia de testing.

En python es con @abstractmethod.

Segun el algoritmo 2.8, el primer paso es:

- Paso 1. Verificar el contrato de unicidad: Dos llamadas a GestorFlota.get_instancia() deben retornar exactamente el mismo objeto. Usa assert instancia1 is instancia2 (identidad no igualdad).

En python, se verifica esto con el operador "is", que comprueba la identidad de los objetos (si apuntan a la misma dirección de memoria).

```
(.venv) PS C:\Users\jason\Downloads\Verificacion-Validacion-sw\Unidad_2_code\python> python -m pytest .\tests\test_ejercicio5.py
===== test session starts =====
platform win32 -- Python 3.14.2, pytest-9.0.3, pluggy-1.6.0
rootdir: C:\Users\jason\Downloads\Verificacion-Validacion-sw\Unidad_2_code\python
plugins: cov-7.1.0
collected 1 item

tests\test_ejercicio5.py . [100%]

===== 1 passed in 0.06s =====
(.venv) PS C:\Users\jason\Downloads\Verificacion-Validacion-sw\Unidad_2_code\python> ls
```

Figura 7: Resultado del test de unicidad de instancia.

Obtuvimos en la figura 7 "pass" que confirma la identidad de la instancia, lo que significa que se ha verificado la propiedad de unicidad del singleton mediante el operador is, garantizando que multiples llamadas a get_instancia() retornan la misma referencia exacta en memoria. Esto cumple con la Definición 2.15 asegurando que la clase no puede instanciarse mas de una vez simultaneamente.

"Def 2.15: Clase que no puede instanciarse directamente. Define métodos abstractos (contrato sin implementación) y puede definir métodos concretos (implementación compartida por todas las subclases)."

6.2. Consigna 22. Implementá un fixture pytest con scope='function' que invoque _reset_para_tests() en el teardown. Explicá por qué es necesario este reset y qué problema de contaminación de estado se evita (Algoritmo 2.8, Paso 3).

- Paso 3. Implementar reset entre tests: al ser un objeto compartido, el estado que deja un test contamina el siguiente. Usar tearDown para restablecer el estado o exponer un método resetForTesting() solo visible en el scope de test.

Siguiendo la Definición 2.5 (fixture de prueba), que establece que un fixture se prepara en el setUp y se desmonta en el tearDown, y aplicando el paso 3 del algoritmo 2.8, implementamos un fixture pytest con scope='function' que invoca el método _reset_para_tests() en el teardown. Esto es necesario para garantizar que cada test se ejecute en un entorno limpio y consistente, evitando la contaminación de estado entre tests. Si no se restablece el estado del singleton entre tests, los cambios realizados por un test podrían afectar el resultado de los tests posteriores, lo que dificulta la identificación de errores y la confiabilidad de los resultados. Al implementar el reset, aseguramos que cada test comience con el mismo estado inicial del singleton, lo que mejora la precisión y la validez de las pruebas.

```
import pytest
from flota.gestor_flota import GestorFlota
```

```
@pytest.fixture(scope="function")
def gestor_reset():
    # fixtura que garantiza el aislamiento entre tests, este scope asegura que se ejecute para cada test

    #arrange | setup: obtenemos la instancia del singleton
    gestor = GestorFlota.get_instancia()
    #uso de yield (ayuda de gemini: yield permite devolver un valor desde un fixture y luego ejecutar el resto del test)
    yield gestor
    #teardown: desmontar el estado del singleton para evitar contaminación entre tests
    GestorFlota._reset_para_tests()
```

#ejecutar con python -m pytest .\tests\test_ej5-p2.py

La necesidad de este reset y el problema que evita se fundamenta con:

- Según la Def 2.1 (estado de un objeto), el estado interno determina el comportamiento observable de los métodos. Al ser entonces singleton una única instancia, cualquier cambio en sus atributos se mantendrá entre llamadas. Sin reset, un test no partiría en un estado conocido, lo que no cumpliría con la INDEPENDENCIA DE LOS CASOS DE PRUEBA.
- Al ser un objeto compartido y además lo anterior que es una única instancia, la contaminación existe en cada test.

El uso de esta fixture permite que la fase de arrange del patrón AAA sea reproducible y controlada, garantizando que el objeto bajo prueba CUT inicie siempre en el estado que se especifica.

6.3. Demostraré el problema de contaminación sin el reset: escribí dos tests donde el segundo falla porque el primero dejó estado acumulado en el singleton. Luego corregí usando el fixture del punto anterior.

De acuerdo a la consigna anterior, sin un reset el estado del objeto persiste entre casos de prueba. Según la definición 2.1, el estado del objeto (sus atributos) condiciona su comportamiento observable futuro.

Para demostrar esto, escribimos dos tests donde el primero modifica el estado del singleton y el segundo test falla porque ese estado modificado afecta su resultado. Luego, corregimos este problema utilizando la fixture con el reset implementado en el punto anterior, asegurando que cada test comience con un estado limpio y evitando la contaminación entre ellos.

Debemos recordar que se exige que cada caso sea independiente y no dependa del orden de ejecución ni del estado por otros tests.

```
PS C:\Users\jason\Downloads\Verificacion-Validacion-sw\Unidad_2_code\python> python -m pytest tests/test_ej5p3.py
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-9.0.3, pluggy-1.6.0
collected 2 items

tests/test_ej5p3.py .F [100%]

===== FAILURES =====
test_B_gestor_vacio

def test_B_gestor_vacio():
    gestor= GestorFlota.get_instancia() # fallara porque el test A ya agrego un vehiculo, entonces el gestor no esta vacio ()
    #arrange: El test asume que el gestor empieza vacio
    # ACT y ASSERT: Al intentar registrar, lanzara ValueError porque la patente
    # ya existe del test anterior, aunque este test no la agrego.
    > gestor.registrar(vehiculo_mock)

tests/test_ej5p3.py:25:
-----
self = <flota.gestor_flota.GestorFlota object at 0x000020C6C088790>, vehiculo = <MagicMock id='2252402164816'>

def registrar(self, vehiculo) -> None:
    if vehiculo.get_patente() in self._vehiculos:
        raise ValueError("Patente ya registrada")
    >
    ValueError: Patente ya registrada

flota\gestor_flota.py:26: ValueError
===== short test summary info =====
FAILED tests/test_ej5p3.py::test_B_gestor_vacio - ValueError: Patente ya registrada
===== 1 failed, 1 passed in 0.15s =====
PS C:\Users\jason\Downloads\Verificacion-Validacion-sw\Unidad_2_code\python>
```

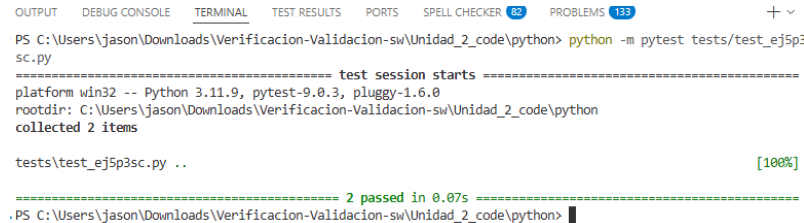
Figura 8: Resultado del test de contaminación de estado.

El resultado de la Figura 8 muestra que el segundo test falla debido a la contaminación de estado causada por el primer test. Esto confirma que sin un reset adecuado, los cambios en el

estado del singleton afectan los resultados de los tests posteriores, lo que viola la independencia de los casos de prueba. Al implementar el fixture con el reset, se garantiza que cada test comience con un estado limpio.

Ahora vamos a hacer una corrección el scope fixture de reset (usamos teardown para establecer el estado).

sabes que Yield es una herramienta mecánica para separar el setup que es lo que está antes del tearDown que es lo de después.



```

OUTPUT  DEBUG CONSOLE  TERMINAL  TEST RESULTS  PORTS  SPELL CHECKER  PROBLEMS
PS C:\Users\jason\Downloads\Verificacion-Validacion-sw\Unidad_2_code\python> python -m pytest tests/test_ej5p3.py
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-9.0.3, pluggy-1.6.0
rootdir: C:\Users\jason\Downloads\Verificacion-Validacion-sw\Unidad_2_code\python
collected 2 items

tests\test_ej5p3.py .. [100%]

===== 2 passed in 0.075s =====
PS C:\Users\jason\Downloads\Verificacion-Validacion-sw\Unidad_2_code\python>

```

Figura 9: Resultado del test de contaminación de estado corregido.

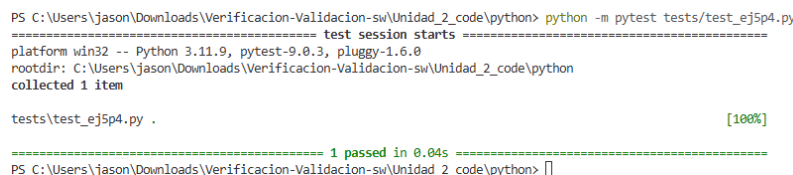
En la Figura 9 se observa que ambos tests pasan correctamente después de implementar el fixture con el reset. Esto demuestra que al restablecer el estado del singleton entre tests, se ha eliminado la contaminación de estado, permitiendo que cada test se ejecute de manera independiente y confiable.

6.4. Verificá thread-safety del Singleton: usá el módulo threading de Python para lanzar 50 hilos que llamen get_instancia() concurrentemente. Verificá que todas las instancias retornadas son idénticas (Algoritmo 2.8, Paso 4).

- Paso 4. Testear thread-safety si el singleton puede ser accedido desde múltiples hilos: verificar que getInstance() retorna el mismo objeto ante accesos concurrentes.

Entonces, para verificar la seguridad en entornos de multihilo, testaremos con el módulo threading de Python, lanzando 50 hilos que llamen a get_instancia() de manera concurrente. Luego, verificaremos que todas las instancias retornadas por estos hilos son idénticas, lo que confirmará que el singleton es thread-safe y que no se crean múltiples instancias en situaciones de acceso concurrente. Si el singleton puede ser accedido desde múltiples hilos, verificar que getInstance() retorna el mismo objeto ante accesos concurrentes.

Lo que hicimos fue: - crear una lista compartida para almacenar las instancias retornadas por cada hilo. - lanzar 50 hilos que ejecuten una función que llama a get_instancia() y almacena el resultado en la lista compartida. - esperar a que todos los hilos terminen su ejecución. - verificar que todas las instancias en la lista son idénticas utilizando el operador 'is'.



```

PS C:\Users\jason\Downloads\Verificacion-Validacion-sw\Unidad_2_code\python> python -m pytest tests/test_ej5p4.py
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-9.0.3, pluggy-1.6.0
rootdir: C:\Users\jason\Downloads\Verificacion-Validacion-sw\Unidad_2_code\python
collected 1 item

tests\test_ej5p4.py . [100%]

===== 1 passed in 0.045s =====
PS C:\Users\jason\Downloads\Verificacion-Validacion-sw\Unidad_2_code\python>

```

Figura 10: Resultado del test de thread-safety del singleton.

En la Figura 10 se muestra que todas las instancias retornadas por los 50 hilos son idénticas, lo que confirma que el singleton es thread-safe. Esto significa que incluso en situaciones de acceso concurrente, get_instancia() retorna la misma instancia del singleton, evitando la creación de múltiples instancias y garantizando la integridad del diseño del singleton en entornos multihilo.

SINGLETON ES THREAD-SAFE

6.5. Consigna 25: Analizá el método `asignar()` del `GestorFlota` desde la perspectiva del testing: ¿es un buen diseño que el Singleton delegue en los objetos `Vehiculo`? ¿Cómo simplifica o complica el testing la existencia del Singleton como intermediario?

El método `testing` `asignar()` del `GestorFlota`, que delega en los objetos `Vehiculo` es un buen diseño por:

```
def asignar(self, patente: str, legajo: str, km: float) -> str:
    # 1.validacion (Precondicion)
    if patente not in self._vehiculos:
        raise KeyError('Vehículo no encontrado')

    # 2. delegacion (El Gestor le pide al Vehiculo que haga el trabajo)
    resultado = self._vehiculos[patente].asignar(legajo, km)

    # 3. cambio de estado (Postcondicion del Gestor)
    self._asignaciones += 1

    return resultado
```

El Vehículo sería el colaborador del `GestorFlota`.

¿Es un buen diseño delegar en los objetos `Vehiculo`?

Si, es un buen diseño para el testing, basado en el paso 3 del algoritmo 2.1

- El algoritmo exige "identificar las dependencias externas (colaboradores) y reemplazarlas con mocks". Al delegar, el `Vehiculo` se vuelve un colaborador independiente. - Podemos usar un `MagicMock` para simular al `Vehiculo`. Esto permite probar que el `GestorFlota` incrementa su contador `_asignaciones` correctamente sin importar si la lógica interna del `Vehiculo` es compleja o falla.

¿Cómo simplifica el testing Singleton?

- Al ser una instancia única, facilita la verificación de los Invariantes de Clase (Definición 2.2). Existe un solo lugar donde verificar que el total de asignaciones de toda la empresa es consistente.

Pero además complica, según la Definición 2.1, el estado (los datos guardados) condiciona el comportamiento futuro. Como el Singleton es global y compartido, el Paso 3 del Algoritmo 2.8 advierte que un test puede "ensuciar" al siguiente.

Para mantener la Independencia de los Tests (Algoritmo 1.1, Paso 6), estamos obligados a implementar un `teardown` que invoque `_reset_para_tests()`. Si un test aumenta el contador `_asignaciones`, el test de abajo verá ese número aumentado y podría fallar erróneamente.

Referencias

- [Bat24] Ned Batchelder. Coverage.py: Code coverage measurement for python. <https://coverage.readthedocs.io/>, 2024. Accedido: 2024.
- [Bin99] J. R. C. L. Binder. Testing object-oriented software: A lifecycle approach. *IBM Systems Journal*, 38(2-3):341–364, 1999.
- [Fou24] Python Software Foundation. Python language reference. <https://docs.python.org/3/>, 2024. Accedido: 2024.
- [Ova24] Cristopher Jason Ovaillos. Repositorio tp validación y verificación. <https://github.com/tu-usuario/repo>, 2024. Código fuente disponible en: /codigo/.
- [pd24a] pytest developers. How to write assertions. <https://docs.pytest.org/en/stable/how-to/assert.html>, 2024. Accedido: 2024.
- [pd24b] pytest developers. pytest fixtures: A tool for test dependency injection. <https://docs.pytest.org/en/stable/fixture.html>, 2024. Accedido: 2024.
- [pd24c] pytest developers. pytest: The python testing framework. <https://docs.pytest.org/>, 2024. Accedido: 2024.

- [Pro24] The LaTeX Project. Latex - a document preparation system. <https://www.latex-project.org/>, 2024. Accedido: 2024.
- [Tan24] Till Tantau. Tikz and pgf. <https://pgf-tikz.github.io/>, 2024. Accedido: 2024.
- [Tea24] JUnit Team. Junit 5 user guide. <https://junit.org/junit5/docs/current/user-guide/>, 2024. Accedido: 2024.
- [vB78] G. von Bochmann. Testing software with a finite state machine model. *IEEE Transactions on Computers*, C-27(12):1130–1139, 1978.